

LAPORAN PRAKTIKUM STRUKTUR DATA
Double Linked List



Oleh:

Naufal Arkan Octyandi

2511533005

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU: DR. WAHYUDI, S.T, M.T

ASISTEN LABOR: SHERLY SUKMADIRA

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 11 MEI 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data yang berjudul “Double Linked List” dengan baik dan tepat waktu. Manusia menemukan cara membuat node saling menunjuk ke depan dan belakang lalu merasa telah menaklukkan semesta. Lumayan menghibur.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum serta untuk menambah wawasan dan pemahaman mengenai konsep struktur data, khususnya penggunaan Double Linked List. Dalam praktikum ini, penulis mempelajari bagaimana cara menyimpan dan mengelola data menggunakan node yang saling terhubung dua arah, serta melakukan operasi dasar seperti penambahan data, penghapusan data, penelusuran maju (forward traversal), dan penelusuran mundur (backward traversal) menggunakan bahasa pemrograman Java.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca dan dapat menjadi referensi tambahan dalam mempelajari struktur data, khususnya konsep Double Linked List.

Padang, 2026

Tim Penyusun

Naufal Arkan Octyandi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 LATAR BELAKANG	1
1.2 TUJUAN PRAKTIKUM	1
1.3 MANFAAT PRAKTIKUM.....	1
BAB II PEMBAHASAN PRAKTIKUM.....	2
2.1 IMPLEMENTASI CLASS NODE PADA DOUBLE LINKED LIST	2
2.2 IMPLEMENTASI PENAMBAHAN NODE PADA DOUBLE LINKED LIST.....	2
2.3 IMPLEMENTASI PENELUSURAN PADA DOUBLE LINKED LIST	5
2.4 IMPLEMENTASI PENGHAPUSAN NODE PADA DOUBLE LINKED LIST.....	6
BAB III KESIMPULAN.....	9
3.1 KESIMPULAN.....	9
DAFTAR PUSTAKA.....	10

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses secara efisien. Dalam pengembangan program, pemilihan struktur data yang tepat sangat berpengaruh terhadap kinerja dan kemudahan dalam pengolahan data. Salah satu struktur data yang sering digunakan adalah Linked List.

Double Linked List merupakan pengembangan dari Single Linked List, di mana setiap node tidak hanya memiliki pointer yang menunjuk ke node berikutnya, tetapi juga memiliki pointer yang menunjuk ke node sebelumnya. Dengan adanya dua arah penunjuk tersebut, proses penelusuran data dapat dilakukan secara maju maupun mundur. Konsep ini membuat Double Linked List lebih fleksibel dalam pengolahan data dibandingkan Single Linked List. Karena tentu manusia merasa satu arah belum cukup rumit, jadi ditambahkanlah pointer kedua supaya debugging makin berwarna.

Dalam praktikum ini, penulis mempelajari implementasi Double Linked List menggunakan bahasa pemrograman Java. Praktikum dilakukan dengan membuat operasi-operasi dasar seperti menambahkan data, menghapus data, menampilkan data secara forward traversal dan backward traversal, serta memahami hubungan antar node dalam struktur Double Linked List.

Melalui praktikum ini diharapkan penulis dapat memahami konsep dasar Double Linked List serta mampu mengimplementasikannya dalam penyelesaian masalah pemrograman yang berkaitan dengan pengelolaan data dinamis.

1.2 TUJUAN PRAKTIKUM

1. Memahami konsep dasar struktur data Double Linked List.
2. Mengetahui perbedaan antara Single Linked List dan Double Linked List.
3. Mengimplementasikan Double Linked List menggunakan bahasa pemrograman Java.

1.3 MANFAAT PRAKTIKUM

1. Menambah pemahaman mengenai struktur data dinamis.
2. Melatih kemampuan pemrograman dalam mengimplementasikan Double Linked List.
3. Mempermudah pemahaman tentang hubungan antar node dua arah.
4. Menjadi dasar dalam mempelajari struktur data yang lebih kompleks.

BAB II

PEMBAHASAN PRAKTIKUM

2.1 IMPLEMENTASI CLASS NODE PADA DOUBLE LINKED LIST

Pertama tama kita membuat class NodeDLL. Class ini digunakan sebagai node pada struktur data Double Linked List. Setiap node memiliki data serta dua pointer, yaitu next untuk menunjuk node berikutnya dan prev untuk menunjuk node sebelumnya, sehingga data bisa ditelusuri maju maupun mundur. Variabel data digunakan untuk menyimpan nilai bertipe integer. Lalu next dan prev digunakan sebagai penghubung antar node. Pada constructor, nilai data diisi sesuai parameter yang diberikan, sedangkan next dan prev diatur menjadi null karena node baru belum terhubung dengan node lain.

```
1 package pekan6_2511533005;
2
3 public class NodeDLL_2511533005 {
4     // mendefinisikan kelas node
5     int data_3005; // data
6     NodeDLL_2511533005 next_3005; // Pointer ke next node
7     NodeDLL_2511533005 prev_3005; // Pointer ke previous node
8
9     // konstruktor
10    public NodeDLL_2511533005(int data) {
11        this.data_3005 = data;
12        this.next_3005 = null;
13        this.prev_3005 = null;
14    }
15 }
```

2.2 IMPLEMENTASI PENAMBAHAN NODE PADA DOUBLE LINKED LIST

Lalu kita membuat class InsertDLL. Class ini digunakan untuk menambahkan node pada Double Linked List, baik di awal, di akhir, maupun di posisi tertentu. Selain itu terdapat method untuk menampilkan isi linked list dan method main untuk menjalankan program. Karena kalau cuma bikin node tanpa dimasukin ke list, itu cuma objek kesepian di memori.

Method insertBegin digunakan untuk menambahkan node di awal linked list. Pertama dibuat node baru, lalu pointer next dari node baru diarahkan ke head lama. Jika head tidak kosong, maka pointer prev dari head lama diarahkan ke node baru. Setelah itu node baru dijadikan head baru.

Method insertEnd digunakan untuk menambahkan node di akhir linked list. Jika linked list masih kosong maka node baru langsung dijadikan head. Jika tidak kosong, program melakukan penelusuran sampai node terakhir menggunakan

perulangan while. Setelah ketemu node terakhir, pointer next node terakhir diarahkan ke node baru dan pointer prev node baru diarahkan ke node terakhir.

Method insertAtPosition digunakan untuk menambahkan node pada posisi tertentu. Jika posisi adalah 1 maka prosesnya sama seperti insert di awal. Jika bukan, program menelusuri linked list sampai posisi sebelum tempat node akan dimasukkan. Setelah itu pointer next dan prev diatur agar node baru tersambung dengan node sebelum dan sesudahnya.

Method printList digunakan untuk menampilkan isi Double Linked List dari depan ke belakang menggunakan perulangan while.

Pada method main, dibuat Double Linked List awal berisi 2 <-> 3 <-> 5. Setelah itu program mencoba menambahkan node 1 di awal, node 6 di akhir, dan node 4 di posisi ke-4, lalu hasilnya ditampilkan menggunakan printList.

```
1 package pe2511533005;
2
3 public class InsertDLL_2511533005 {
4     // menambahkan node diawal DLL
5     static NodeDLL_2511533005 insertBegin (NodeDLL_2511533005 head, int data) {
6         // buat node baru
7         NodeDLL_2511533005 new_node_3005 = new NodeDLL_2511533005(data);
8         // jadikan pointer nextnya head
9         new_node_3005.next_3005 = head;
10        // jadikan pointer prev head ke new_node
11        if (head != null) {
12            head.prev_3005 = new_node_3005;
13        }
14        return new_node_3005;
15    }
16    // fungsi menambahkan node diakhir
17    public static NodeDLL_2511533005 insertEnd (NodeDLL_2511533005 head, int newData) {
18        // buat node baru
19        NodeDLL_2511533005 newNode_3005 = new NodeDLL_2511533005(newData);
20        // jika dll null jadikan head
21        if (head == null) {
22            head = newNode_3005;
23        }
24        else {
25            NodeDLL_2511533005 curr_3005 = head;
26            while (curr_3005.next_3005 != null) {
27                curr_3005 = curr_3005.next_3005;
28            }
29            curr_3005.next_3005 = newNode_3005;
30            newNode_3005.prev_3005 = curr_3005;
31        }
32        return head;
33    }
34    // fungsi menambahkan node di posisi tertentu
35    public static NodeDLL_2511533005 insertAtPosition (NodeDLL_2511533005 head, int pos, int new_data) {
36        // buat node baru
37        NodeDLL_2511533005 new_node_3005 = new NodeDLL_2511533005 (new_data);
38        if (pos == 1) {
39            new_node_3005.next_3005 = head;
40            if (head != null) {
```

```

36 // buat node baru
37 NodeDLL_2511533005 new_node_3005 = new NodeDLL_2511533005 (new_data);
38 if (pos == 1) {
39     new_node_3005.next_3005 = head;
40     if (head != null) {
41         head.prev_3005 = new_node_3005;
42     }
43     head = new_node_3005;
44     return head;
45 }
46 NodeDLL_2511533005 curr_3005 = head;
47 for (int i = 1; i < pos -1 && curr_3005 != null; ++i) {
48     curr_3005 = curr_3005.next_3005;
49 }
50 if (curr_3005 == null) {
51     System.out.println("Posisi tidak ada");
52     return head;
53 }
54 new_node_3005.prev_3005 = curr_3005;
55 new_node_3005.next_3005 = curr_3005.next_3005;
56 curr_3005.next_3005 = new_node_3005;
57 if (new_node_3005.next_3005 != null) {
58     new_node_3005.next_3005.prev_3005 = new_node_3005;
59 }
60 return head;
61 }
62 public static void printList (NodeDLL_2511533005 head) {
63     NodeDLL_2511533005 curr_3005 = head;
64     while (curr_3005 != null) {
65         System.out.print(curr_3005.data_3005 + " <-> ");
66         curr_3005 = curr_3005.next_3005;
67     }
68     System.out.println();
69 }
70 public static void main(String[] args) {
71     // membuat dll 2 <-> 3 <-> 5
72     NodeDLL_2511533005 head_3005 = new NodeDLL_2511533005(2);
73     head_3005.next_3005 = new NodeDLL_2511533005(3);
74     head_3005.next_3005.prev_3005 = head_3005;
75     head_3005.next_3005.next_3005 = new NodeDLL_2511533005(5);

```

```

59     }
60     return head;
61 }
62 public static void printList (NodeDLL_2511533005 head) {
63     NodeDLL_2511533005 curr_3005 = head;
64     while (curr_3005 != null) {
65         System.out.print(curr_3005.data_3005 + " <-> ");
66         curr_3005 = curr_3005.next_3005;
67     }
68     System.out.println();
69 }
70 public static void main(String[] args) {
71     // membuat dll 2 <-> 3 <-> 5
72     NodeDLL_2511533005 head_3005 = new NodeDLL_2511533005(2);
73     head_3005.next_3005 = new NodeDLL_2511533005(3);
74     head_3005.next_3005.prev_3005 = head_3005;
75     head_3005.next_3005.next_3005 = new NodeDLL_2511533005(5);
76     head_3005.next_3005.next_3005.prev_3005 = head_3005.next_3005;
77     // cetak DLL awal
78     System.out.print("DLL Awal: ");
79     printList(head_3005);
80     // tambah 1 di awal
81     head_3005 = insertBegin(head_3005, 1);
82     System.out.print("simpul 1 ditambah awal: ");
83     printList(head_3005);
84     // head 6 diakhir
85     System.out.print("simpul 6 ditambah di akhir: ");
86     int data_3005 = 6;
87     head_3005 = insertEnd(head_3005, data_3005);
88     printList(head_3005);
89     // menambahkan node 4 di posisi 4
90     System.out.print("tambah node 4 di posisi 4: ");
91     int data2_3005 = 4;
92     int pos_3005 = 4;
93     head_3005 = insertAtPosition (head_3005, pos_3005, data2_3005);
94     printList(head_3005);
95 }
96 }
97 }
98

```


2.3 IMPLEMENTASI PENELUSURAN PADA DOUBLE LINKED LIST

Selanjutnya kita membuat class PenelusuranDLL. Class ini digunakan untuk melakukan penelusuran data pada Double Linked List, baik secara maju maupun mundur. Hal ini bisa dilakukan karena setiap node memiliki pointer next dan prev. Jadi node bisa bolak-balik seperti manusia yang ragu milih jalan hidup.

Method forwardTraversal digunakan untuk menelusuri linked list dari depan ke belakang. Penelusuran dimulai dari head, lalu program menggunakan perulangan while untuk mencetak data setiap node dan berpindah ke node berikutnya menggunakan pointer next sampai mencapai null.

Method backwardTraversal digunakan untuk menelusuri linked list dari belakang ke depan. Penelusuran dimulai dari tail, lalu data setiap node dicetak sambil berpindah ke node sebelumnya menggunakan pointer prev sampai mencapai null.

Pada method main, pertama dibuat Double Linked List berisi node 1 <-> 2 <-> 3. Setelah semua node saling terhubung menggunakan pointer next dan prev, program menjalankan forwardTraversal untuk menampilkan data dari depan ke belakang dan backwardTraversal untuk menampilkan data dari belakang ke depan.

```
1 package pekan6_2511533005;
2
3 public class PenelusuranDLL_2511533005 {
4     // fungsi penelusuran maju
5     static void forwardTraversal (NodeDLL_2511533005 head) {
6         // memulai penelusuran dari head
7         NodeDLL_2511533005 curr_3005 = head;
8         // lanjutkan sampai akhir
9         while (curr_3005 != null) {
10             // print data
11             System.out.print(curr_3005.data_3005 + " <-> ");
12             // pindah ke node berikutnya
13             curr_3005 = curr_3005.next_3005;
14         }
15         // print spasi
16         System.out.println();
17     }
18     // fungsi penelusuran mundur
19     static void backwardTraversal (NodeDLL_2511533005 tail) {
20         // mulai dari akhir
21         NodeDLL_2511533005 curr_3005 = tail;
22         // lanjut sampai head
23         while (curr_3005 != null) {
24             // cetak data
25             System.out.print(curr_3005.data_3005 + " <-> ");
26             // pindah ke node sebelumnya
27             curr_3005 = curr_3005.prev_3005;
28         }
29         // cetak spasi
30         System.out.println();
31     }
32     public static void main(String[] args) {
33         // cetak DLL
34         NodeDLL_2511533005 head_3005 = new NodeDLL_2511533005(1);
35         NodeDLL_2511533005 second_3005 = new NodeDLL_2511533005(2);
36         NodeDLL_2511533005 third_3005 = new NodeDLL_2511533005(3);
37
38         head_3005.next_3005 = second_3005;
39         second_3005.prev_3005 = head_3005;
40         second_3005.next_3005 = third_3005;
```

```

13         curr_3005 = curr_3005.next_3005;
14     }
15     // print spasi
16     System.out.println();
17 }
18 // fungsi penelusuran mundur
19 static void backwardTraversal (NodeDLL_2511533005 tail) {
20     // mulai dari akhir
21     NodeDLL_2511533005 curr_3005 = tail;
22     // lanjut sampai head
23     while (curr_3005 != null) {
24         // cetak data
25         System.out.print(curr_3005.data_3005 + " <-> ");
26         // pindah ke node sebelumnya
27         curr_3005 = curr_3005.prev_3005;
28     }
29     // cetak spasi
30     System.out.println();
31 }
32 public static void main(String[] args) {
33     // cetak DLL
34     NodeDLL_2511533005 head_3005 = new NodeDLL_2511533005(1);
35     NodeDLL_2511533005 second_3005 = new NodeDLL_2511533005(2);
36     NodeDLL_2511533005 third_3005 = new NodeDLL_2511533005(3);
37
38     head_3005.next_3005 = second_3005;
39     second_3005.prev_3005 = head_3005;
40     second_3005.next_3005 = third_3005;
41     third_3005.prev_3005 = second_3005;
42
43     System.out.println("Penelusuran maju: ");
44     forwardTraversal (head_3005);
45
46     System.out.println("Penelusuran mundur: ");
47     backwardTraversal (third_3005);
48 }
49 }
50 }
51 }
52 }

```

2.4 IMPLEMENTASI PENGHAPUSAN NODE PADA DOUBLE LINKED LIST

Selanjutnya kita membuat class HapusDLL. Class ini digunakan untuk menghapus node pada Double Linked List, baik di awal, di akhir, maupun di posisi tertentu. Selain itu terdapat method untuk mencetak isi linked list dan method main untuk menjalankan program. Karena setelah belajar menambah data, manusia pasti lanjut belajar menghapusnya. Siklus kehidupan node.

Method delHead digunakan untuk menghapus node paling awal atau head. Jika linked list kosong maka program mengembalikan null. Jika tidak kosong, head dipindahkan ke node berikutnya menggunakan pointer next, lalu pointer prev dari head baru diubah menjadi null.

Method delLast digunakan untuk menghapus node terakhir. Jika linked list kosong atau hanya memiliki satu node maka hasilnya null. Jika tidak, program menelusuri linked list sampai node terakhir menggunakan perulangan while. Setelah itu pointer next dari node sebelumnya diubah menjadi null agar node terakhir terputus dari linked list.

Method delPos digunakan untuk menghapus node pada posisi tertentu. Program terlebih dahulu menelusuri linked list sampai posisi yang akan dihapus. Jika posisi

ditemukan, pointer next dan prev dari node sekitar akan diatur ulang agar node tersebut terlepas dari linked list. Jika node yang dihapus adalah head, maka head dipindahkan ke node berikutnya.

Method printList digunakan untuk menampilkan isi Double Linked List dari depan ke belakang menggunakan perulangan while.

Pada method main, pertama dibuat Double Linked List berisi 1 2 3 4 5. Setelah itu program mencoba menghapus node di awal menggunakan delHead, menghapus node terakhir menggunakan delLast, dan menghapus node posisi ke-2 menggunakan delPos, lalu hasilnya ditampilkan menggunakan printList.

```
1 package pekan6_2511533005;
2
3 public class HapusDLL_2511533005 {
4     // fungsi menghapus node awal
5     public static NodeDLL_2511533005 delHead (NodeDLL_2511533005 head) {
6         if (head == null) {
7             return null;
8         }
9         NodeDLL_2511533005 temp_3005 = head;
10        head = head.next_3005;
11        if (head != null) {
12            head.prev_3005 = null;
13        }
14        return head;
15    }
16    //fungsi menghapus diakhir
17    public static NodeDLL_2511533005 dellast (NodeDLL_2511533005 head) {
18        if (head == null) {
19            return null;
20        }
21        if (head.next_3005 == null) {
22            return null;
23        }
24        NodeDLL_2511533005 curr_3005 = head;
25        while (curr_3005.next_3005 != null) {
26            curr_3005 = curr_3005.next_3005;
27        }
28        // update pointer previous node
29        if (curr_3005.prev_3005 != null) {
30            curr_3005.prev_3005.next_3005 = null;
31        }
32        return head;
33    }
34    // fungsi menghapus node posisi tertentu
35    public static NodeDLL_2511533005 delPos(NodeDLL_2511533005 head, int pos) {
36        //jika DLL kosong
37        if(head == null) {
38            return null;
39        }
40        NodeDLL_2511533005 curr_3005 = head;
```

```

35 public static NodeDLL_2511533005 delPos(NodeDLL_2511533005 head, int pos) {
36     //jika DLL kosong
37     if(head == null) {
38         return null;
39     }
40     NodeDLL_2511533005 curr_3005 = head;
41     //telusuri sampai ke node yang akan dihapus
42     for (int i = 1; curr_3005 != null && i < pos; ++i) {
43         curr_3005 = curr_3005.next_3005;
44     }
45     //jika posisi tidak ditemukan
46     if (curr_3005 == null) {
47         return head;
48     }
49     //update pointer
50     if (curr_3005.prev_3005 != null) {
51         curr_3005.prev_3005.next_3005 = curr_3005.next_3005;
52     }
53     if (curr_3005.next_3005 != null) {
54         curr_3005.next_3005.prev_3005 = curr_3005.prev_3005;
55     }
56     //jika tang dihapus head
57     if (head == curr_3005) {
58         head = curr_3005.next_3005;
59     }
60     return head;
61 }
62 //fungsi mencetak DLL
63 public static void printList (NodeDLL_2511533005 head) {
64     NodeDLL_2511533005 curr_3005 = head;
65     while (curr_3005 != null) {
66         System.out.print(curr_3005.data_3005 + " ");
67         curr_3005 = curr_3005.next_3005;
68     }
69     System.out.println();
70 }
71 public static void main(String[] args) {
72     //buat sebuah DLL
73     NodeDLL_2511533005 head_3005 = new NodeDLL_2511533005(1);
74     head_3005.next_3005 = new NodeDLL_2511533005(2);

```

```

63 public static void printList (NodeDLL_2511533005 head) {
64     NodeDLL_2511533005 curr_3005 = head;
65     while (curr_3005 != null) {
66         System.out.print(curr_3005.data_3005 + " ");
67         curr_3005 = curr_3005.next_3005;
68     }
69     System.out.println();
70 }
71 public static void main(String[] args) {
72     //buat sebuah DLL
73     NodeDLL_2511533005 head_3005 = new NodeDLL_2511533005(1);
74     head_3005.next_3005 = new NodeDLL_2511533005(2);
75     head_3005.next_3005.prev_3005 = head_3005;
76     head_3005.next_3005.next_3005 = new NodeDLL_2511533005(3);
77     head_3005.next_3005.next_3005.prev_3005 = head_3005.next_3005;
78     head_3005.next_3005.next_3005.next_3005 = new NodeDLL_2511533005(4);
79     head_3005.next_3005.next_3005.next_3005.prev_3005 = head_3005.next_3005.next_3005;
80     head_3005.next_3005.next_3005.next_3005.next_3005 = new NodeDLL_2511533005(5);
81     head_3005.next_3005.next_3005.next_3005.next_3005.prev_3005 = head_3005.next_3005.next_3005.next_3005;
82
83     System.out.print("DLL Awal: ");
84     printList(head_3005);
85
86     System.out.print("Setelah head dihapus: ");
87     head_3005 = delHead(head_3005);
88     printList(head_3005);
89
90     System.out.print("Setelah node terakhir dihapus: ");
91     head_3005 = delLast(head_3005);
92     printList(head_3005);
93
94     System.out.print("Menghapus node ke 2: ");
95     head_3005 = delPos(head_3005, 2);
96
97     printList(head_3005);
98 }
99 }
100
101 }
102

```

BAB III

KESIMPULAN

3.1 KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List merupakan struktur data dinamis yang memiliki dua pointer pada setiap node, yaitu pointer next dan prev. Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan secara maju maupun mundur sehingga pengelolaan data menjadi lebih fleksibel dibandingkan Single Linked List.

Dalam praktikum ini, penulis berhasil mengimplementasikan beberapa operasi dasar pada Double Linked List menggunakan bahasa pemrograman Java, seperti menambahkan node di awal, akhir, dan posisi tertentu, melakukan penelusuran maju dan mundur, serta menghapus node pada bagian tertentu. Dari implementasi tersebut dapat dipahami bagaimana hubungan antar node bekerja melalui pointer yang saling terhubung.

Praktikum ini juga membantu penulis memahami konsep struktur data dinamis serta meningkatkan kemampuan logika pemrograman dalam mengelola data menggunakan Double Linked List. Pada akhirnya, sebagian besar waktu praktikum memang dihabiskan untuk memastikan pointer tidak salah sambung dan program tidak menghasilkan null. Tradisi yang terus diwariskan dalam dunia linked list.

DAFTAR PUSTAKA

Programiz - Doubly Linked List Data Structure

[Doubly Linked List \(With code\)](#)

GeeksforGeeks - Doubly Linked List Introduction

[Doubly Linked List Tutorial - GeeksforGeeks](#)